

DAEDALUS

Core Engine — Pure Java Maze / Simulation Engine

:: Generators • 9 Solvers • Grid Model • Stats • Heuristics ::

Module: daedalus-core

Dependencies: SLF4J API only — zero Spring, zero JavaFX, zero servlet

Contract: Perfect mazes are spanning trees (property test enforces this)

Embeddable: Can be dropped into any JVM host (desktop, server, LoadBalancerPro, etc.)

1. Engine Architecture

The core is deliberately minimal and framework-free. It provides the mathematical foundation for the entire Daedalus stack: a cell-graph maze model, generator SPI, solver SPI, and rich runtime metrics.

Layer	Key Classes	Purpose
Model	Point, Cell, Direction, MazeGrid, MazeStats, MazeMetadata	Immutable grid coordinates + mutable cell state + metrics
Generator SPI	MazeGenerator (interface)	Contract: produce perfect spanning-tree mazes
Solver SPI	MazeSolver + AbstractMazeSolver + 9 implementations	Pathfinding from start to goal with stats
Heuristics	Heuristics (MANHATTAN, EUCLIDEAN, CHEBYSHEV + tie-breaker)	Plug-in A*/IDA* distance functions
Registry	SolverRegistry	Central lookup for built-in + plugin solvers

2. Perfect Maze Property (Enforced by Test)

Every generator must produce a **spanning tree**: every cell reachable, no cycles, exactly $\text{rows} \times \text{cols} - 1$ carved edges. The `PerfectMazePropertyTest` (12×17 grid, seed 42) uses flood-fill + BFS cycle detection + edge counting to verify this contract.

2.1 Test Assertions

```
// 1. Connectivity – every cell reachable from start
Set<Point> reachable = floodFill(grid, start);
assertThat(reachable).hasSize(totalCells);

// 2. Acyclicity – no back-edges in BFS parent tree
assertThat(hasNoCycles(grid, start)).isTrue();

// 3. Edge count – tree on N nodes has exactly N-1 edges
int carvedEdges = countCarvedEdges(grid);
assertThat(carvedEdges).isEqualTo(totalCells - 1);
```

This single test is the **foundation of correctness** for the entire engine. Any generator that violates it will fail CI.

3. Solver Catalog (9 Implementations)

Solver	id	Optimal?	Memory	Notes
BFS	bfs	Yes (unweighted)	$O(V)$	Shortest path, layer-by-layer
DFS	dfs	No	$O(V)$	Fast but suboptimal
A*	astar	Yes (admissible h)	$O(b^d)$	Best-first with $f = g + h$
IDA*	ida-star	Yes	$O(d)$	A* optimality + DFS memory
Dijkstra	dijkstra	Yes (weighted)	$O((V+E) \log V)$	Generalizes BFS to weights
Bidirectional	bidirectional	Yes	$O(b^{(d/2)})$	Two frontiers meet in middle
Trémaux	tremaux	No	$O(E)$	Mark passages 0/1/2 — classic 19th c. method
Dead-End Filling	dead-end-filling	Yes on perfect	$O(V)$	Fill dead-ends then BFS skeleton
Wall Follower	wall-follower	No	$O(1)$	Right-hand rule — hugs walls

All solvers extend `AbstractMazeSolver`` (provides `reconstruct(parent, start, goal)``). They populate `MazeStats`` (visited, explored, frontier size, path length, success) for live visualization and complexity analysis.

4. Core Model & Heuristics

4.1 Point (Immutable Record)

```
public record Point(int row, int col) {  
    public Point translate(int dr, int dc) { ... }  
    public Point step(Direction d) { ... }  
    public int manhattan(Point other) { ... }  
    public double euclidean(Point other) { ... }  
}
```

4.2 Heuristics Catalog

```
public static final ToDoubleBiFunction<Point, Point> MANHATTAN =  
    (a, b) -> Math.abs(a.row() - b.row()) + Math.abs(a.col() - b.col());  
  
public static final ToDoubleBiFunction<Point, Point> EUCLIDEAN = Point::euclidean;  
  
public static final ToDoubleBiFunction<Point, Point> CHEBYSHEV =  
    (a, b) -> Math.max(Math.abs(a.row() - b.row()), Math.abs(a.col() - b.col()));  
  
// Tie-breaker for A*/IDA* (small bias toward straight lines)  
public static ToDoubleBiFunction<Point, Point> manhattanWithTieBreaker(double epsilon) { ... }
```

4.3 MazeStats (Runtime Metrics)

Every generation or solve populates `MazeStats`. This powers live solver visualizers, leaderboard scoring, and the `theory.ComplexityAnalyzer` (not shown in this batch but referenced in Javadoc).

5. How Core Fits in the Stack

daedalus-core is the **only** module with no framework dependencies. Everything else builds on it:

Consumer	Uses From Core
daedalus-plugin-runtime	Registers generators/solvers via MazePlugin#registerAlgorithms(ctx)
daedalus-server	MazeGenerationService, MazeSolverService, REST endpoints, STOMP frames
daedalus-desktop	Theme-aware canvas renderer + solver visualizer (pulls stats live)
External hosts	Can embed core directly (e.g. LoadBalancerPro, custom tools)

This completes the full Daedalus documentation set. You now have professional PDF references for Server, Plugin Runtime, Desktop, and Core Engine — plus the TypeScript DTOs from the first response. The entire stack is documented and ready for extension or deployment.